

Steering the Lorenz Attractor

A mathematical summary of the method implemented in this repository

Summary in five points

1. **The plant.** The controlled object is the Lorenz system with a scalar forcing u entering additively in the $\dot{x}$ equation, integrated by RK4 at fixed step $\Delta t = 0.005$, with all horizons measured in Lyapunov times $\tau = 1/\lambda_1$, $\lambda_1 \approx 0.9056$.
2. **The objective.** The control goal — keep the trajectory in the positive- x lobe of the attractor — is a hard, non-differentiable indicator $\mathbf{1}[x > 0]$, which is relaxed into a smooth surrogate $\varphi(x) = \frac{1}{2}(1 + \tanh(x/\varepsilon))$ whose time-averaged shortfall $\langle 1 - \varphi(x) \rangle$ is the task loss actually differentiated.
3. **The policy and its gradient.** The controller is a linear state-feedback law $u = w \cdot s + b$ whose four parameters are learned by backpropagating the loss *through* the integrator — a discrete adjoint over the closed-loop flow — and updated with Adam.
4. **The horizon obstruction.** Because the linearized flow of a chaotic system amplifies perturbations like $e^{t/\tau}$, the pathwise gradient $\partial \mathcal{L} / \partial(w, b)$ grows exponentially in the length of the training window, which bounds usable training to $O(1)$ Lyapunov time and makes the training horizon the central experimental axis of the project.
5. **The effort penalty and its moment closure.** A quadratic control-effort term $\lambda \langle (u/u_{\text{ref}})^2 \rangle$ bounds the actuation, and because u is *linear* in the state it collapses exactly to a quadratic form $w^\top M w + 2b m^\top w + b^2$ in the first two time-averaged state moments — letting the penalty be evaluated over the full deployment horizon with an analytic, exponentially-stable gradient while the trajectory itself is held outside the autograd graph.

1 The plant: a forced Lorenz system in Lyapunov time

The state is $s = (x, y, z) \in \mathbb{R}^3$ and the dynamics are the classical Lorenz equations with a single scalar actuator injected into the first component:

$$\dot{x} = \sigma(y - x) + u, \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z, \quad (1)$$

with the canonical chaotic parameter set $\sigma = 10, \rho = 28, \beta = 8/3$. The choice to force only $\dot{x}$ makes the problem genuinely underactuated: one input against a three-dimensional state, with the y and z channels reachable only indirectly through the nonlinear coupling $x(\rho - z)$ and xy .

Time is discretized at a fixed step $\Delta t = 0.005$, small enough that the integrator rather than the step size is the accuracy-limiting choice. Two integrators are implemented. Forward Euler,

$s_{n+1} = s_n + \Delta t f(s_n, u(s_n))$, is retained for comparison, but the default is classical fourth-order Runge–Kutta. The important structural detail is that the control is re-evaluated at every RK4 stage rather than frozen across the step:

$$k_i = f\left(s^{(i)}, u(s^{(i)})\right), \quad s_{n+1} = s_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \quad (2)$$

This means the scheme integrates the *closed-loop* vector field $s \mapsto f(s, u(s))$ to fourth order, rather than integrating the open-loop field under a piecewise-constant hold. The distinction matters because the object being differentiated during training is precisely the closed-loop flow map; a stage-frozen control would make the discrete system a different dynamical system from the one the gradient describes, and would degrade the scheme to first order in the feedback path.

The unifying quantity for the whole project is the leading Lyapunov exponent $\lambda_1 \approx 0.9056$ of the unforced attractor, and its reciprocal, the Lyapunov time $\tau = 1/\lambda_1 \approx 1.10$. Every horizon in the code is specified in Lyapunov times H and converted to a step count by

$$N = \text{round}\left(\frac{H}{\lambda_1 \Delta t}\right), \quad (3)$$

so that a training window of $H = 1$ is exactly one e -folding time of infinitesimal perturbations, and the default evaluation window of $H = 100$ is one hundred of them — some 22,085 integration steps. Expressing horizons this way is not cosmetic: it is what makes the central result of the project (how far training can see before its gradients dissolve) a statement about the *dynamics* rather than about an arbitrary choice of Δt . Plots inherit the same convention, with all time axes labeled in t/τ .

2 The objective: a smooth surrogate for a discontinuous goal

The control task is a *symbolic* one: the Lorenz attractor has two lobes, one at $x > 0$ and one at $x < 0$, and the free system switches between them aperiodically. The goal is to bias the trajectory to spend as much time as possible in the positive lobe. The natural figure of merit is therefore the occupation fraction

$$S = \frac{1}{N+1} \sum_{n=0}^N \mathbf{1}[x_n > 0], \quad (4)$$

the *success fraction*, and this is the number reported for every run and plotted against every swept parameter.

This quantity is useless as a training signal. The indicator $\mathbf{1}[x > 0]$ is piecewise constant, so $\partial S / \partial x_n$ is zero almost everywhere and undefined on the switching set: no gradient information whatsoever reaches the controller. The standard remedy is applied — replace the hard indicator with a smooth sigmoidal relaxation. The code uses the tanh form

$$\varphi_\varepsilon(x) = \frac{1}{2} \left(1 + \tanh \frac{x}{\varepsilon}\right), \quad (5)$$

with softness $\varepsilon = 2$, and defines the **task loss** as the time-averaged shortfall from full occupancy:

$$\mathcal{L}_{\text{task}} = \frac{1}{N+1} \sum_{n=0}^N (1 - \varphi_\varepsilon(x_n)). \quad (6)$$

Several properties of this relaxation are worth stating explicitly. First, it is a genuine surrogate: $\varphi_\varepsilon \rightarrow \mathbf{1}[x > 0]$ pointwise as $\varepsilon \rightarrow 0^+$, so $\mathcal{L}_{\text{task}} \rightarrow 1 - S$, and minimizing the loss is asymptotically maximizing the metric. Second, its derivative $\varphi'_\varepsilon(x) = \frac{1}{2\varepsilon} \text{sech}^2(x/\varepsilon)$ is strictly positive everywhere,

so *every* state along the trajectory contributes gradient — including states deep in the wrong lobe, which a hard loss would leave flat.

Third, ε is a real modelling choice rather than a numerical detail. Since the attractor’s lobes sit at $|x| \sim 10\text{--}15$, a softness of $\varepsilon = 2$ makes φ effectively saturated on both lobes and steeply varying only in the narrow transit region near $x \approx 0$. This concentrates the learning signal exactly where the topological decision is made — at the lobe crossings — which is the dynamically meaningful place for a one-dimensional actuator to intervene. A much larger ε would smear the objective into “make x large”, a quantitatively different and easier problem; a much smaller one would recover the vanishing gradients of the hard indicator.

The separation between the differentiated surrogate and the reported metric is maintained deliberately throughout: the soft loss is what the optimizer sees, the hard occupation fraction is what the results claim. No run is ever scored on the quantity it was trained on.

3 The policy and its gradient: differentiating through the flow

The controller is deliberately the simplest nontrivial one: a static linear state-feedback law

$$u(s) = w \cdot s + b = w_1x + w_2y + w_3z + b, \quad (7)$$

with parameters $\theta = (w, b) \in \mathbb{R}^4$, initialized at zero so that training begins from the uncontrolled system. The restriction to a linear law is a modelling commitment, not a limitation of convenience: it keeps the closed-loop system a *polynomial* vector field of the same quadratic degree as the original Lorenz system, so the controlled dynamics remain an object one can reason about analytically, and — as §5 exploits — it makes the effort cost exactly quadratic in θ .

Training is direct trajectory optimization. Composing N integrator steps gives a discrete flow map Φ_θ^N carrying the initial condition s_0 to the whole trajectory $\{s_n\}_{n=0}^N$, each s_n being a differentiable function of θ through every preceding step. The objective

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{task}}(\{s_n(\theta)\}) + \lambda \mathcal{L}_{\text{effort}}(\theta) \quad (8)$$

is then minimized by gradient descent, with $\nabla_\theta \mathcal{L}$ obtained by reverse-mode automatic differentiation through the unrolled integrator. This is *discretize-then-optimize*: the gradient computed is the exact gradient of the discrete objective, not a discretization of the continuous adjoint, so it is consistent with the trajectory that is actually simulated.

Structurally, the backward pass is the discrete adjoint recursion. Writing $J_n = \partial s_{n+1} / \partial s_n$ for the Jacobian of one closed-loop RK4 step and $B_n = \partial s_{n+1} / \partial \theta$ for its explicit parameter dependence, the adjoint state $a_n = \partial \mathcal{L} / \partial s_n$ satisfies the backward sweep

$$a_n = J_n^\top a_{n+1} + \frac{\partial \ell_n}{\partial s_n}, \quad \nabla_\theta \mathcal{L} = \sum_n B_n^\top a_{n+1}, \quad (9)$$

where ℓ_n is the per-step contribution to the loss. The gradient is therefore an accumulation of products of closed-loop Jacobians — which is exactly the structure that §4 shows to be the method’s binding constraint. Because the control is re-evaluated at each RK4 stage, J_n is the Jacobian of the true closed-loop step, so the adjoint is self-consistent with the forward dynamics.

The parameter update is Adam with bias correction, using first and second moment estimates $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. Adam is a reasonable default here for a specific reason: the four parameters have very different natural scales — w_3 multiplies $z \sim 25$ while b acts alone — and the per-coordinate normalization $\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ makes the effective step size scale-free in each coordinate, which a single global learning rate on raw gradients would not. Training runs a fixed iteration budget (600 by default) with no early stopping, and the loss history is recorded as the triple (task, penalty, total) so that the two competing terms can be read separately.

4 The horizon obstruction: exponential gradient blow-up

The defining difficulty of this problem, and the reason the training window is the project’s primary independent variable, is that differentiating through a chaotic flow is unstable in a precise and quantifiable sense.

Consider the sensitivity of a late state to an early perturbation. Along a trajectory, an infinitesimal perturbation δs_0 evolves under the linearized flow, $\delta s(t) = D\Phi^t \delta s_0$, and the defining property of a chaotic attractor is that the leading singular value of $D\Phi^t$ grows exponentially at the rate set by the top Lyapunov exponent:

$$\|D\Phi^t\| \sim e^{\lambda_1 t} = e^{t/\tau}. \quad (10)$$

Since the adjoint recursion of §3 accumulates precisely the transposed products $J_n^\top J_{n+1}^\top \cdots$, the loss gradient inherits this growth directly:

$$\|\nabla_\theta \mathcal{L}_{\text{task}}\| \sim e^H \quad (11)$$

for a training window of H Lyapunov times. The repository measures both halves of this claim. A single-parameter probe differentiates the terminal coordinate $x(t)$ with respect to a constant forcing u and plots $|\partial x(t)/\partial u|$ against horizon on a log axis, where it tracks the reference curve $e^{t/\tau}$; a second diagnostic re-initializes the policy at each horizon and plots $\|\nabla_\theta \mathcal{L}_{\text{task}}\|$ over windows out to eight Lyapunov times, which grows by many orders of magnitude. A complementary view of the same phenomenon integrates two copies of the free system under constant forcings differing by $\Delta u = 1$ and plots the separation of their velocity fields $\|\dot{s}_1 - \dot{s}_2\|$, which is straight on a semilog axis over the pre-saturation range.

The consequence is a hard trade-off with no purely numerical escape. The gradient does not merely become large — largeness alone could be absorbed by Adam’s normalization — it becomes *uninformative*. Beyond a few Lyapunov times the computed derivative is dominated by the amplification of the particular trajectory’s finite-time stretching directions and by accumulated floating-point error, so it describes the sensitivity of one realization rather than any statistically meaningful property of the attractor. It is, in the project’s own phrasing, amplified noise. Meanwhile a *shorter* window has the opposite defect: it optimizes a loss that says nothing about behaviour past its own end, so a policy tuned over one Lyapunov time has no direct guarantee over the hundred it is evaluated on.

This is why success-versus-training-horizon is the canonical figure of the project, and why the training window is marked as a swept axis alongside the effort weight, the learning rate, and the initial condition. The empirical question the sweeps answer is where on that trade-off the optimum sits: how far the window can be pushed before gradient corruption outweighs the extra foresight, and how sharply performance falls away on either side. Notably, this obstruction applies *only* to the pathwise task term — the effort term is engineered in the next section to be free of it entirely.

5 The effort penalty and its moment closure

Left alone, the task objective is unbounded in the wrong direction. Nothing in $\mathcal{L}_{\text{task}}$ constrains the magnitude of u , so the optimizer’s cheapest strategy is to drive $|w|$ upward and hold x positive by brute force — a law that “succeeds” by demanding physically absurd actuation. The regularized objective is therefore the default:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \left\langle \left(\frac{u}{u_{\text{ref}}} \right)^2 \right\rangle, \quad (12)$$

a standard quadratic effort cost with reference scale $u_{\text{ref}} = 60$ (chosen so the penalty is $O(1)$ at a control amplitude comparable to the attractor’s own scales) and weight $\lambda = 0.07$ by default. The angle brackets denote a time average, and the control at the final state is excluded, since it is never applied.

The interesting design question is *over which window* to measure the effort. Averaging over the short training window prices only the control expended during the optimization horizon, and a law that looks cheap over one Lyapunov time can cost several times more once the run continues to its full deployment length. The honest thing is to price the control actually deployed — that is, to average over the long evaluation horizon. But that appears to reintroduce exactly the problem of §4: differentiating a 22,000-step rollout would produce a gradient amplified by e^{100} , and would require storing every intermediate state.

The linearity of the policy resolves this exactly. Since $u_n = w \cdot s_n + b$,

$$\langle u^2 \rangle = \langle (w \cdot s + b)^2 \rangle = w^\top \langle ss^\top \rangle w + 2b \langle s \rangle^\top w + b^2 = w^\top M w + 2b m^\top w + b^2, \quad (13)$$

where $M = \langle ss^\top \rangle \in \mathbb{R}^{3 \times 3}$ and $m = \langle s \rangle \in \mathbb{R}^3$ are the first two time-averaged moments of the closed-loop trajectory. This is a **moment closure**: the entire dependence of the penalty on the trajectory enters through nine plus three numbers. Everything else about the states is irrelevant to this term.

The algorithmic payoff is substantial. The long rollout is performed with the policy parameters *detached* — it leaves the automatic-differentiation graph entirely — and the moments (M, m) are treated as constants. The gradient then reaches θ analytically through the quadratic form alone,

$$\nabla_w \langle u^2 \rangle = 2(Mw + bm), \quad \partial_b \langle u^2 \rangle = 2(m^\top w + b), \quad (14)$$

which is exact, well-conditioned, and completely free of exponential amplification. Memory is flat in the number of steps rather than linear, since only running sums are accumulated. And the value and gradient obtained this way are *identical* to those of the pathwise penalty evaluated over the same frozen states — the closure is an exact reformulation, not an approximation of the objective it replaces.

The two regimes are nonetheless distinct objectives: when the effort window coincides with the training window, the penalty is differentiated pathwise through the states, and the gradient includes the term describing how changing θ moves the trajectory itself. When the windows differ, the states are frozen and that term is dropped by construction. Runs of the two kinds are kept strictly separate rather than mixed.

Constants of the method

Symbol	Meaning	Value
σ	Prandtl number	10
ρ	Rayleigh number	28
β	geometric factor	8/3
Δt	integration step	0.005
λ_1	leading Lyapunov exponent	0.9056
τ	Lyapunov time, $1/\lambda_1$	1.104
ε	indicator softness	2
u_{ref}	effort reference scale	60
λ	effort weight	0.07
H_{train}	training window	1τ
H_{eval}	evaluation window	100τ
N_{eval}	evaluation steps	22,085
—	training iterations	600